

API REST d'accès aux données

Xavier André & Romain Tavenard

1 Rappel : organisation de votre code

Pour ce TD, vous créerez un nouveau fichier `td7.py`. Dans ce fichier, votre code sera organisé de la manière suivante :

```
# Section 1 : Imports de module
import requests

# Section 2 : Définition de fonctions
def [...]
```

Section 3 : Tests de fonctions définies et manipulations en mode "script"

```
if __name__ == "__main__":
    pass # Remplacer par vos appels de test
```

Notamment, vous définirez vos fonctions en début de fichier et les appels seront listés en fin de fichier, **à l'intérieur du bloc `if __name__ == "__main__":`**. De cette manière, vous pourrez, d'une question à l'autre, réutiliser les fonctions déjà codées au besoin.

Bonne pratique : pour chaque requête HTTP, appelez `reponse.raise_for_status()` juste après `requests.get(...)`. Cela lève immédiatement une exception si le serveur répond avec un code d'erreur (404, 500...), plutôt que de laisser le code planter silencieusement plus loin.

```
reponse = requests.get(url, params={...})
reponse.raise_for_status()
data = reponse.json()
```

2 Énoncé

1. Se rendre sur le [site de la STAR](#) et trouver l'API indiquant la position des bus de la STAR en temps réel. Cliquez sur l'onglet "API" pour accéder aux options de requête. Observez les valeurs que peut prendre l'attribut `etat`.
2. Combien de résultats (`results`) retourne la requête proposée par défaut ? Et combien de résultats sont contenus dans la base (attribut `total_count`) ? Comment faire pour vous assurer d'avoir tous les résultats lorsque vous formulez une requête ?

Pour les questions suivantes, il est conseillé d'importer le module `pprint` qui permet d'afficher de manière claire les dictionnaires :

```
from pprint import pprint

[...]
pprint(mon_joli_dictionnaire)
```

3. Écrire une fonction qui retourne une liste des bus **en service** sur le réseau, chaque bus étant représenté par un dictionnaire qui contient les clés `numerobus`, `nomcourtligne`, `destination`, `ecartsecondes` et `position`. Pour cela, consultez l'interface d'édition de requêtes de l'API de la STAR (celle que vous avez trouvé à la question 1) et ajoutez la condition `refine` pour que `etat` vaille "`En ligne`", puis notez l'URL générée (clic droit sur le lien du bas de la page, puis "Copier le lien").
4. Écrire une fonction qui retourne le nombre de bus en avance (qui ont un attribut `ecartsecondes` positif) dans le jeu de données.
5. Écrire une fonction qui retourne, pour chaque ligne de bus, une liste des `ecartsecondes` des bus de cette ligne.

3 Lien avec le TD6 : enrichissement des données restaurants via une API

Dans le TD6, vous avez manipulé le fichier `NYfood.json` contenant des restaurants new-yorkais. Nous allons maintenant enrichir ces données en interrogeant l'[API Open Data NYC](#) qui expose le même jeu de données (les inspections sanitaires des restaurants de New York) en ligne.

L'URL de base de l'API est : `https://data.cityofnewyork.us/resource/43nn-pn8j.json`

6. Écrire une fonction `fetch_restaurants_par_type` qui prend en entrée un type de cuisine (`cuisine`) et retourne la liste des restaurants de ce type exposés par l'API (attribut

`cuisinedescription`), en limitant à 50 résultats (ajout de la paire clé-valeur `$limit=50` aux paramètres passés à `requests.get`). Chaque élément de la liste retournée sera un dictionnaire avec au minimum les clés `camis` (identifiant), `dba` (nom du restaurant) et `zipcode`.

Assurez-vous d'utiliser `raise_for_status()` dans votre fonction.

7. Écrire une fonction `fetch_score_moyen_par_cuisine` qui prend en entrée une liste de types de cuisine et retourne un dictionnaire associant chaque type au score moyen (`score`) des restaurants de ce type dans l'API. Les restaurants sans score seront ignorés.

Testez votre fonction avec les types `["Italian", "Chinese", "French"]`.

8. En chargeant le fichier `NYfood.json` du TD6 et en interrogeant l'API, écrire une fonction qui compare, pour chaque type de cuisine présent dans votre fichier local, le nombre de restaurants dans le fichier local et le nombre de restaurants retournés par l'API (toujours limité à 50). Affichez les résultats sous la forme :

```
Italian : 12 restaurants locaux, 50 restaurants API
Chinese : 30 restaurants locaux, 50 restaurants API
...
```